DOSSIER DE TRANSMISSION • SPÉCIFICATION + MODE OPÉRATOIRE

MT4 MT5
LIBRARY MANAGER PRO

DOSSIER COMPLET POUR LE DÉVELOPPEUR

Construire une véritable application macOS installable, pilotable et développée avec Claude Code, avec un accompagnement pas à pas depuis Android avant l'accès au Mac.

Version du dossier : 3.0 • Référence produit : cahier des charges V2 + guide opératoire consolidés

À qui s'adresse ce document : à la personne chargée de réaliser le logiciel. Le document contient à la fois le besoin métier, les exigences techniques, le plan de développement, les critères de recette et le mode opératoire pour piloter Claude Code depuis un téléphone Android.

Contexte utilisateur : le propriétaire du projet est débutant en programmation. Il dispose actuellement d'un téléphone Android et aura accès à un Mac ultérieurement. Le développeur doit donc rendre le travail pilotable à distance et organiser la livraison Mac en deux temps.

Décisions verrouillées : périmètre ALL ; analyse SET maximum ; nettoyage par quarantaine restaurable ; capacité cible 50 000 fichiers minimum ; originaux en lecture seule pendant l'analyse.

Exigence absolue : aucune maquette présentée comme un logiciel fini. Toute fonctionnalité doit être réellement implémentée, testée et traçable. Aucun original ne doit être modifié pendant l'analyse.

0. Résumé exécutif et objectifs

Construire MT4 MT5 Library Manager PRO, une application de bureau macOS destinée à inventorier, comprendre, rapprocher, dédupliquer et organiser une grande bibliothèque de fichiers MetaTrader 4 et MetaTrader 5. Le cas d'usage initial est une collection d'environ 6 000 robots, avec des doublons, des homonymes de versions différentes, des fichiers SET désorganisés et des associations robot/SET inconnues.

- Analyser tous les fichiers, y compris extensions inconnues et fichiers annexes.
- Reconnaître et classer les éléments MetaTrader dans la limite de ce que leurs formats permettent réellement.
- Analyser les SET au maximum et suggérer les associations avec explication et score de confiance.
- Distinguer doublons binaires exacts, fichiers similaires et versions différentes.
- Créer une bibliothèque organisée par copie vérifiée, sans altérer les sources.
- Mettre en quarantaine uniquement les doublons exacts sûrs, avec restauration et manifeste.
- Gérer 50 000 fichiers ou davantage avec reprise, annulation, journaux et mémoire maîtrisée.

1. Décisions produit non négociables

Décision	Valeur retenue	Conséquence
Périmètre	ALL	Scanner tous types de fichiers et conserver les extensions inconnues.
Analyse SET	Maximum	Parser, normaliser, comparer, détecter orphelins et suggérer les correspondances avec preuves.
Nettoyage	Quarantine	Aucune suppression définitive automatique ; quarantaine réversible et journalisée.
Échelle	50 000+ fichiers	Traitement en flux, indexation, pagination, base locale et tests de charge.
Protection des sources	Lecture seule	Aucune écriture, renommage, déplacement ou suppression dans les dossiers sources pendant les analyses.
Plateforme	macOS	Livraison .app et DMG ; compatibilité testée, non supposée.
Pilotage	Android à distance, Mac plus tard	Dépôt Git privé et développement cloud/à distance ; build final sur Mac.

2. Utilisateurs, périmètre et exclusions

Utilisateurs et scénarios

- Utilisateur non technique : lance les scans, lit les rapports, valide les associations et les opérations de quarantaine.
- Développeur : maintient le code, les tests, les migrations, les builds et la documentation.
- Scénario A : importer un ou plusieurs dossiers source en lecture seule et inventorier leur contenu.
- Scénario B : retrouver les SET qui correspondent à un robot, avec justification.
- Scénario C : identifier les doublons exacts et les versions distinctes.
- Scénario D : prévisualiser une bibliothèque organisée puis copier les fichiers vers une destination.
- Scénario E : mettre en quarantaine des doublons exacts après simulation et confirmation, puis restaurer si nécessaire.

Exclusions explicites

- Ne jamais exécuter les EA, indicateurs, scripts, DLL ou exécutables analysés.
- Ne pas promettre la décompilation d'EX4/EX5 ou l'extraction de paramètres qui ne sont pas accessibles.
- Ne pas supprimer automatiquement les versions différentes, fichiers similaires ou associations incertaines.
- Ne pas modifier les originaux pour les renommer, corriger, déplacer ou convertir.
- Ne pas prétendre à la compatibilité avec toutes les versions macOS sans matrice et preuves de tests.

3. Exigences fonctionnelles détaillées

PHASE F1 — Gestion des sources et inventaire

Comportement attendu : L'utilisateur ajoute des dossiers à analyser ; le logiciel inventorie récursivement sans modifier leur contenu.

Fonctionnement technique : Utiliser des accès fichiers en lecture seule autant que possible. Enregistrer les racines sources, chemins relatifs, extension, taille, dates disponibles, statut et erreurs. Gérer chemins longs, Unicode, liens symboliques et permissions refusées sans boucle infinie.

Critères de validation : Tests : dossiers imbriqués, noms accentués, extensions inconnues, permissions refusées, liens symboliques cycliques, fichier inaccessible. Les hashes avant/après doivent rester identiques.

PHASE F2 — Métadonnées et empreintes

Comportement attendu : Le logiciel collecte les métadonnées accessibles et calcule une empreinte stable pour comparer les fichiers.

Fonctionnement technique : SHA-256 en flux par blocs ; ne jamais charger un fichier complet en RAM. Distinguer hash non calculé, calcul en cours, erreur et hash validé. Enregistrer taille, dates, extension, chemin, attributs accessibles et version du moteur d'analyse.

Critères de validation : Tests : fichiers vides, gros fichiers, lecture interrompue, fichier modifié pendant le scan, reprise et cohérence du hash.

PHASE F3 — Analyse des formats et archives

Comportement attendu : Identifier les formats connus et inspecter les archives sans exécuter leur contenu.

Fonctionnement technique : Détecter par extension et signature binaire lorsque possible. ZIP au minimum ; limites configurables de profondeur, taille décompressée, ratio de compression et nombre d'entrées. Rejeter traversal de chemin, symlinks dangereux et archives suspectes.

Critères de validation : Tests : archive saine, corrompue, zip bomb simulée, noms traversants, doublons d'entrées, archive chiffrée. L'analyse doit échouer proprement et journaliser.

PHASE F4 — Reconnaissance MetaTrader

Comportement attendu : Classer les fichiers EA, indicateurs, scripts, sources, SET, DLL et ressources, avec niveau de certitude.

Fonctionnement technique : Reconnaissance par extensions, signatures et métadonnées disponibles. MQ4/MQ5 lisibles comme sources texte ; EX4/EX5 considérés comme binaires compilés opaques sauf métadonnées vérifiables. Ne jamais les exécuter.

Critères de validation : Tests : exemples fictifs MT4/MT5, fichiers renommés, extension trompeuse, binaire inconnu. L'interface doit afficher les limites de reconnaissance.

PHASE F5 — Analyse SET maximale

Comportement attendu : Lire et structurer les paramètres des SET, puis comparer les paramètres aux robots/sources pour proposer les correspondances.

Fonctionnement technique : Parser les lignes clé=valeur, commentaires, encodages courants, booléens, nombres, chaînes, suffixes de paramètres si présents. Normaliser sans perdre la valeur originale. Comparer noms de paramètres, types, valeurs par défaut et indices de source. Score explicable et pondéré, avec preuves et incertitudes.

Critères de validation : Tests : SET vide, mal formé, encodages, doublons de clés, paramètres inconnus, variantes MT4/MT5, SET presque identiques. Une association non prouvée reste « à vérifier ».

PHASE F6 — Familles et versions

Comportement attendu : Regrouper les robots et fichiers associés tout en conservant les versions distinctes.

Fonctionnement technique : Créer une identité logique séparée de l'empreinte binaire. Exploiter nom, métadonnées, paramètres, source et contenu accessible. Afficher la raison de regroupement et le niveau de confiance.

Critères de validation : Tests : même nom/contenu différent, version dans le nom, version dans les métadonnées, compilation différente, copie identique renommée.

PHASE F7 — Détection des doublons

Comportement attendu : Séparer strictement doublons binaires exacts et fichiers similaires.

Fonctionnement technique : Préfiltrer par taille, puis comparer SHA-256 ; ne déclarer exact qu'après vérification du hash et stabilité des fichiers. Similarité sémantique ou de texte séparée, jamais équivalente à identité binaire.

Critères de validation : Tests : copie exacte, fichier presque identique, homonyme différent, même contenu renommé, collision théorique traitée par vérification supplémentaire si nécessaire.

PHASE F8 — Quarantaine et restauration

Comportement attendu : Proposer une simulation, puis déplacer ou copier en quarantaine uniquement les doublons exacts admissibles après confirmation.

Fonctionnement technique : Créer manifeste transactionnel : chemin source, destination, hash, taille, date, décision, horodatage et état. Vérifier hash de la copie avant de marquer l'opération réussie. Prévoir restauration avec vérification de conflit et journal d'audit. Aucun bouton de purge automatique.

Critères de validation : Tests : dry-run, confirmation, disque plein, interruption, destination existante, hash différent, restauration, conflit de chemin. Tout échec doit préserver une copie vérifiée.

PHASE F9 — Organisation de la bibliothèque

Comportement attendu : Créer une proposition d'arborescence et une copie organisée de la bibliothèque.

Fonctionnement technique : Règles configurables MT4/MT5, type, famille, version, statut SET et inconnus. Prévisualisation avant copie. Copie vérifiée par hash, conservation des originaux et rapport des collisions de noms.

Critères de validation : Tests : collision de nom, reprise de copie, destination indisponible, espace insuffisant, hash non conforme.

PHASE F10 — Recherche, filtres et rapports

Comportement attendu : Permettre de rechercher, filtrer et exporter l'inventaire, les correspondances, les doublons et les erreurs.

Fonctionnement technique : Index local SQLite ou solution équivalente ; recherche paginée, filtres multi-critères, export CSV/JSON et rapports lisibles. Les valeurs affichées doivent provenir des données analysées, jamais de valeurs fictives.

Critères de validation : Tests : pagination, caractères Unicode, filtres combinés, export complet, volumes élevés.

PHASE F11 — Interface utilisateur

Comportement attendu : Fournir une interface claire, accessible à un débutant, où chaque action est reliée à un moteur réel.

Fonctionnement technique : Écrans : tableau de bord, sources, scan, explorateur, détail fichier, SET, familles/versions, doublons, quarantaine/restauration, organisation, rapports, paramètres et aide. Afficher progression, pause, reprise, annulation et erreurs.

Critères de validation : Tests UI : parcours utilisateur, états vides, erreurs, annulation, accessibilité clavier, aucune action décorative.

PHASE F12 — Performance, sécurité et fiabilité

Comportement attendu : Supporter au moins 50 000 fichiers sans saturation et sans toucher aux sources.

Fonctionnement technique : Traitement incrémental, tâches en arrière-plan, base indexée, cache contrôlé, transactions, reprise après fermeture, limites mémoire et journaux rotatifs. Protection contre archives hostiles et chemins dangereux.

Critères de validation : Tests de charge reproductibles à 50 000 puis 100 000 fichiers synthétiques. Rapports : durée, débit, RAM max, erreurs, temps de reprise. Les seuils cibles sont à proposer puis faire valider.

PHASE F13 — Livraison macOS

Comportement attendu : Produire une application Mac installable et documenter la compatibilité réellement testée.

Fonctionnement technique : Build .app et DMG. Étudier Intel, Apple Silicon et Universal2 selon dépendances. Signature/notarisation si distribution hors environnement local. Documenter version minimale macOS et limites des outils.

Critères de validation : Tests sur les versions et architectures accessibles. Ne pas annoncer une compatibilité non testée. Fournir procédure d'installation, mise à jour, désinstallation et sauvegarde.

4. Modèle de données minimal

Entité	Champs essentiels
SourceRoot	id, chemin racine, statut lecture seule, date d'ajout, dernier scan
FileRecord	id, chemin relatif, taille, dates, extension, signature, hash, statut, erreurs
FileAnalysis	file_id, type détecté, plateforme MT4/MT5/inconnu, métadonnées extraites, confiance
SetParameter	file_id, nom, valeur brute, valeur normalisée, type estimé, commentaire
Association	robot_id, set_id, score, preuves, incertitudes, validation utilisateur
FileFamily	id logique, membres, relation, confiance, version estimée
DuplicateGroup	hash, membres, type exact/similaire, candidat de référence
QuarantineEntry	source, destination, hash, manifeste, état, date, raison
ScanJob	id, racines, état, progression, reprise, annulation, erreurs
AuditEvent	acteur, action, horodatage, objet, résultat, détails

5. Architecture technique attendue

Le développeur doit proposer l'architecture et justifier les choix avant de coder. Une architecture modulaire est attendue : couche interface, services métier, analyseurs de formats, moteur d'indexation, persistance, sécurité et livraison.

- Séparer le moteur métier de l'interface pour tester les fonctions sans GUI.
- Base locale transactionnelle, migrations versionnées et sauvegardes.
- Scanner asynchrone avec files de tâches bornées et contrôle de concurrence.
- Hash et copie par flux ; jamais de chargement complet des gros fichiers.
- Analyseurs isolés et robustes : un fichier malformé ne doit pas arrêter tout le scan.
- Interface macOS avec intégration propre aux dialogues de sélection de dossiers et permissions.
- Journal d'audit append-only pour les opérations sensibles.

Choix de technologie : à décider après audit. Le développeur doit comparer au minimum une solution native macOS (Swift/SwiftUI) et une solution multiplateforme (par exemple Python/PySide ou autre) en tenant compte des dépendances, performances, compatibilité macOS, signature, maintenance et facilité de build.

6. Sécurité et protection des données

- Les sources sont ouvertes en lecture seule ; l'application ne doit pas demander de permission d'écriture inutile sur les dossiers sources.
- Aucun contenu analysé ne doit être exécuté, interprété comme script système ou chargé comme bibliothèque.
- Les archives sont inspectées avec des limites de sécurité strictes.
- La quarantaine et la bibliothèque organisée doivent être sur des destinations distinctes des sources.
- Chaque opération de quarantaine est atomique autant que possible, vérifiée par hash et journalisée.
- En cas de doute, permission insuffisante, changement de fichier, manque d'espace ou hash incohérent : arrêter et demander une intervention.
- Pas de télémétrie ou transfert cloud des noms, chemins ou contenus de robots sans consentement explicite.

7. Exigences de qualité : aucune maquette

Une fonctionnalité n'est acceptée que si les éléments ci-dessous existent et sont vérifiés.

- Description du comportement attendu et cas d'utilisation.
- Description du fonctionnement technique et des composants touchés.
- Entrées, formats acceptés, préconditions et limites.
- Sorties, données persistées et éléments affichés.
- Erreurs possibles, messages utilisateur et stratégie de reprise.
- Tests unitaires, tests d'intégration et tests UI si pertinents.
- Résultats réels des tests, commande exécutée et environnement.
- Critères d'acceptation observables et démonstration avec données fictives.

Les boutons désactivés doivent être explicitement signalés « non disponible » ; aucun bouton ne doit simuler un succès. Les tests doivent échouer si une fonction n'est pas branchée.

8. Plan de développement et portes de validation

Phase	Livrable	Validation obligatoire
0 — Audit	Architecture, choix techniques, compatibilité et risques	Approbation écrite avant code métier.
1 — Socle	Dépôt, configuration, logs, base et migrations	Tests de base, installation reproductible.
2 — Scanner	Inventaire universel en lecture seule	Jeu test, extensions inconnues, compteurs exacts.
3 — Métadonnées	SHA-256, dates, tailles et reprise	Hashes vérifiés, originaux inchangés.
4 — Archives	Inspection sécurisée	Tests archives malformées/hostiles.
5 — MetaTrader	Reconnaissance MT4/MT5	Classement et limites explicites.
6 — SET	Parser et moteur de correspondance	Preuves, score, tests d'ambiguïté.
7 — Versions	Familles et variantes	Homonymes et versions distincts.
8 — Doublons	Détection et quarantaine	Dry-run et restauration testés.
9 — Organisation	Prévisualisation et copie	Hash destination conforme.
10 — Interface	Écrans et interactions réelles	Parcours complets testés.

Phase	Livrable	Validation obligatoire
11 — Performance	Charge et sécurité	50k/100k synthétiques, rapport mesuré.
12 — Mac	App, DMG, signature et matrice	Installation et tests sur Mac réels.

9. Travail à distance depuis Android avant l'accès au Mac

Le propriétaire du projet souhaite piloter le développement depuis Android. Le développeur doit mettre en place une méthode simple et sécurisée, par exemple Claude Code sur le Web ou environnement distant compatible, relié à un dépôt GitHub privé. Les fonctionnalités disponibles dépendent du compte et de l'environnement choisis.

Procédure de mise en place

- Créer ou utiliser un compte GitHub ; créer un dépôt privé nommé mt4-mt5-library-manager-pro.
- Ne jamais placer les robots, SET, DLL, données de trading, secrets ou fichiers clients dans le dépôt.
- Connecter Claude Code au dépôt selon les options disponibles dans le compte et autoriser uniquement ce dépôt.
- Ajouter les deux documents de spécification à la documentation du dépôt.
- Créer une branche par phase ou fonctionnalité ; conserver les changements validés dans l'historique Git.
- Faire exécuter les tests dans l'environnement distant ; conserver rapports et logs comme artefacts du dépôt ou de la session.
- Expliquer au propriétaire chaque action sur mobile : bouton, écran, texte à copier, résultat attendu.
- Ne jamais prétendre avoir connecté GitHub ou lancé un test si cela n'a pas réellement été fait.

Prompt maître à utiliser dans Claude Code

Je suis le développeur chargé de réaliser MT4 MT5 Library Manager PRO. Le propriétaire est débutant et pilote le projet depuis Android ; il aura accès à un Mac plus tard.

Lis intégralement ce dossier de spécification et traite-le comme référence contractuelle. Décisions verrouillées : périmètre ALL, analyse SET maximum, nettoyage par quarantaine restaurable, 50 000 fichiers minimum, originaux en lecture seule, application macOS.

Commence uniquement par la Phase 0 : audit du dépôt, ambiguïtés, architecture, comparaison des choix techniques, stratégie de compatibilité macOS, plan de tests et risques. N'écris aucun code métier avant validation.

Règles permanentes :
- Chaque fonctionnalité doit documenter comportement, technique, entrées, sorties, erreurs, logs, tests et critères d'acceptation.
- Aucune maquette ou simulation ne peut être présentée comme un moteur terminé.
- Aucune donnée réelle de robots ou SET dans GitHub.
- N'exécute jamais de fichiers MetaTrader analysés.
- Lecture seule sur les sources ; hash en flux ; 50k+ avec mémoire bornée.
- Distinction stricte entre doublons exacts, similaires et versions différentes.
- Quarantaine restaurable avec manifeste, vérification et audit ; aucune suppression définitive automatique.
- Les associations SET/robot doivent montrer preuves, confiance et incertitudes.
- Après chaque phase, lancer les tests disponibles, afficher les commandes et résultats réels, indiquer ce qui exige un Mac, puis attendre l'accord.
- Pour le propriétaire, expliquer simplement et proposer une seule action mobile à la fois.

Maintenant, livre uniquement l'audit Phase 0 et attends validation.

10. Guide opératoire simplifié pour le propriétaire Android

Étape	Action du propriétaire	Ce qu'il doit voir
1	Créer/ouvrir Claude et GitHub sur Android.	Comptes accessibles.
2	Créer le dépôt GitHub privé.	Dépôt privé vide.
3	Envoyer le dossier PDF et le prompt maître au développeur.	Le développeur confirme la lecture et pose les questions restantes.
4	Le développeur connecte Claude Code au dépôt privé.	Accès limité au dépôt confirmé.
5	Valider le plan Phase 0.	Architecture et phases écrites.
6	Valider chaque phase séparément.	Tests réels et rapport à chaque étape.
7	Conserver les rapports et versions validées.	Historique Git et rapports disponibles.

Étape	Action du propriétaire	Ce qu'il doit voir
8	Quand le Mac est disponible, reprendre la Phase 12.	Build Mac, DMG et tests sur la machine réelle.

Messages simples à utiliser

Explique-moi cette étape sans jargon. Dis-moi exactement où appuyer sur Android et quel texte envoyer. Donne-moi une seule action à la fois.

Fais le bilan de la phase : code modifié, tests réellement lancés, résultats, erreurs restantes, éléments qui attendent le Mac. Ne commence pas la phase suivante.

Je ne comprends pas cette commande. Explique ce qu'elle fait, quels fichiers elle modifie et si elle peut toucher à mes originaux avant de me demander de l'exécuter.

11. Livraison finale attendue

- Dépôt source privé avec historique et instructions de build.
- Application macOS .app installable et image DMG.
- Builds Intel et Apple Silicon si supportés par la pile retenue ; Universal2 si toutes les dépendances le permettent.
- Matrice de compatibilité macOS : version, architecture, statut, preuve de test et limites.
- Documentation utilisateur en français, guide d'installation, mise à jour, sauvegarde et restauration.
- Rapport de tests fonctionnels, sécurité, performance 50 000+ fichiers et non-modification des sources.
- Jeu de données synthétiques reproductible.
- Manifeste et procédure de restauration de quarantaine.
- Liste explicite des limitations des formats binaires MetaTrader opaques.

12. Définition de « terminé »

Le projet n'est terminé que lorsque l'application s'installe et se lance sur les versions macOS annoncées, que les moteurs métier sont réellement reliés à l'interface, que les tests de charge et de sécurité sont réussis, que les originaux restent inchangés et que les opérations de quarantaine sont restaurables.

Livrable non acceptable : maquette d'interface, boutons sans action, données de démonstration présentées comme réelles, tests non exécutés, compatibilité inventée, suppression irréversible ou association SET/robot sans justification.

Priorité absolue : fiabilité, traçabilité et protection de la bibliothèque avant vitesse de développement ou apparence de l'interface.

Annexe — Informations de démarrage à fournir au développeur

Le propriétaire doit compléter ces informations lorsqu'elles seront disponibles :

- Version exacte de macOS et modèle de Mac utilisé pour les tests finaux.
- Architecture du Mac : Intel ou Apple Silicon.
- Emplacement des originaux et de la sauvegarde indépendante (à communiquer de manière sécurisée, pas dans un dépôt public).
- Exemples fictifs de SET et de sources MQ4/MQ5 pour les tests.
- Préférence de technologie après présentation des avantages et inconvénients.
- Versions macOS minimales réellement souhaitées, à arbitrer selon faisabilité.